

Adopter Portal API Reference

This document defines the adopter-facing endpoints delivered in Sprint 3: profile management, government ID verification, adoption applications, visit scheduling, favorites, and the adopted-pet health passport view. It follows the same conventions established in 08-Public_API_Reference — see that document for base URL, standard response structure, and error code definitions, which are not repeated here.

1. Profile Endpoints

1.1 GET /api/v1/adopters/me

Returns the full profile of the currently logged-in adopter, including all schema fields from the ADOPTER table except those explicitly excluded for security.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "adopterID": 12,
    "adopterName": "...",
    "adopterEmail": "...",
    "avatarSeed": "a1b2c3d4-...",
    "housingType": "...",
    "ownsOrRents": "...",
    "householdSize": 3,
    "...": "...all other ADOPTER fields except excluded ones"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Adopter record does not exist	NOT_FOUND	404

Excluded fields: adopterPassword and stripeCustomerID are never included in this response, regardless of role — this mirrors the NF-10 data privacy requirement. avatarSeed IS included — it is not sensitive, and the frontend needs it to render the adopter's avatar via the shared DiceBear config (see Avatar Picker spec).

1.2 PUT /api/v1/adopters/me

Allows the logged-in adopter to update their profile fields. Accepts a partial body — only fields present in the request are updated. Password changes are out of scope for this endpoint.

Request

Field	Value
Method	PUT
Auth required	Yes — Adopter only
Body fields	Any subset of updatable ADOPTER fields (see design note)

Enum fields are validated against their allowed values before the update is applied: housingType, ownsOrRents, employmentStatus, activityLevel, preferredSize, preferredAgeRange, adopterType. An invalid enum value returns VALIDATION_ERROR before any write occurs.

Response

```
200 OK
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "...": "same shape as GET /adopters/me" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Invalid enum value in body	VALIDATION_ERROR	422
Attempt to update a disallowed field	VALIDATION_ERROR	422
Adopter record does not exist	NOT_FOUND	404

Disallowed fields: adopterEmail, adopterPassword, adopterRiskFlag, preQualifyFlag, and accountStatus cannot be updated through this endpoint — these are either admin-controlled, security-sensitive, or reserved for the AI matcher (Sprint 6). Any of these present in the request body are silently ignored, not written. avatarSeed IS an allowed field — a string, up to 64 characters, no additional validation beyond standard length/type checking (any string is valid input to the DiceBear generator).

2. Government ID Verification

These endpoints handle identity document upload and status retrieval for adoption-critical identity verification (Requirement A-01). Files are stored in Supabase Storage; only metadata is persisted in the GOVERNMENT_ID table.

2.1 POST /api/v1/adopters/me/government-id

Uploads a government ID document for identity verification. Stores file metadata in the GOVERNMENT_ID table; the file itself is uploaded to the government-ids/ bucket in Supabase Storage.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Content-Type	multipart/form-data
Body fields	idType (string, required), idNumber (string, required, ≤45 chars), file (JPEG/PNG/WebP/HEIC/PDF, ≤5 MB, required)

Response

```
201 Created
{
  "success": true,
  "message": "Government ID submitted successfully",
  "data": {
    "governmentIDID": 7,
    "userID": 12,
    "userType": "Adopter",
    "idType": "Driver's License",
    "idNumber": "*****4567",
    "verificationStatus": "Pending"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing idType, idNumber, or file	VALIDATION_ERROR	422
File exceeds 5 MB or wrong type	BAD_REQUEST	400
Government ID already submitted for this adopter	CONFLICT	409

*Response masking: idNumber is masked in the response (e.g. *****4567) — this is a deliberate current design choice, not a placeholder for a future fix. Revisit only if a product requirement emerges for the adopter to view their full submitted ID number.*

Uniqueness enforcement: duplicate submissions are prevented at the database level via a unique constraint — `@@unique([userID, userType])` — on `GOVERNMENT_ID`, not solely by an application-level pre-check. A pre-check (`findFirst`) may still run first as an optimization to avoid an unnecessary file upload in the common case, but it is not the source of truth for the 409 — the Prisma unique-constraint error (P2002) on the insert is. This closes a race condition where two near-simultaneous requests could both pass a pre-check before either had written its row. On a P2002 error, the just-uploaded Storage file is deleted before the 409 response is returned, to avoid leaving an orphaned file.

2.2 GET /api/v1/adopters/me/government-id

Returns the current adopter's government ID verification status.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via <code>req.user.userID</code>

Response

```
200 OK
{
  "success": true,
  "message": "Government ID status retrieved successfully",
  "data": {
    "idType": "Driver's License",
    "verificationStatus": "Pending",
    "submittedAt": "2026-08-01T10:15:00Z"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
No government ID submitted yet	NOT_FOUND	404

Response scope: `idNumber` and the Storage file URL/path are never returned by this endpoint, regardless of role — only `idType`, `verificationStatus`, and submission date are exposed. This is stricter than the POST response, which masks but still includes `idNumber`.

3. Adoption Applications

3.1 POST /api/v1/adoption-applications

Submits a new adoption application for a specific pet. Application status starts as Pending and is later updated by shelter staff (see PATCH /adoption-applications/:id/status, Sprint 4).

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Body fields	petID (int, required), shelterID (int, required)

shelterID in the request body is expected to match the pet's current shelterID — it identifies which branch the application is being processed at, consistent with the ADOPTION_APPLICATION schema.

Response

```
201 Created
{
  "success": true,
  "message": "Application submitted successfully",
  "data": {
    "applicationID": 44,
    "petID": 9,
    "adopterID": 12,
    "shelterID": 3,
    "applicationStatus": "Pending",
    "createdAt": "2026-08-03T09:00:00Z"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing petID or shelterID	VALIDATION_ERROR	422
Pet does not exist	NOT_FOUND	404
Pet adoptionStatus is not 'available'	CONFLICT	409
Adopter already has an active application for this pet	CONFLICT	409

Duplicate application guard: prevented at the database level via a partial unique index — CREATE UNIQUE INDEX ON ADOPTION_APPLICATION (adopterID, petID) WHERE applicationStatus IN ('Pending', 'Accepted') — not solely by an application-level pre-check. The pre-check may still run first as an early-exit optimization, but the Prisma unique-constraint error (P2002) on the insert is the actual guarantee, following the same check-then-insert-with-DB-backstop pattern used on GOVERNMENT ID. Because the index is scoped to active statuses only, an adopter may resubmit a new application for the same pet once a prior one reaches Rejected or Withdrawn — those are terminal states, not reopened, and a resubmission creates a new applicationID rather than editing the old row.

3.2 GET /api/v1/adoption-applications/:id

Returns a single adoption application by ID. Adopters may only view their own applications; shelter staff may only view applications belonging to their own shelter branch.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter, Shelter Staff
Path params	id (int, required) — applicationID

Ownership is checked after existence: a 404 is returned before any 403 check runs, so a nonexistent applicationID never leaks whether it would have belonged to someone else. For staff, the JWT does not carry shelterID (see 06-Auth_System_Reference) — the handler re-fetches the staff member's current shelterID from the STAFF table before comparing.

Response

```
200 OK
{
  "success": true,
  "message": "Application retrieved successfully",
  "data": {
    "applicationID": 44,
    "petID": 9,
    "adopterID": 12,
    "staffID": null,
    "shelterID": 3,
    "applicationStatus": "Pending",
    "createdAt": "2026-08-03T09:00:00Z",
    "pet": { "petName": "Buddy" },
    "shelter": { "shelterName": "..." }
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Role not adopter/staff	FORBIDDEN	403
Adopter viewing another adopter's application	FORBIDDEN	403
Staff viewing another shelter's application	FORBIDDEN	403
applicationID does not exist	NOT_FOUND	404

3.3 GET /api/v1/adopters/me/applications

Returns a paginated list of the current adopter's own adoption applications, most recent first.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Query params	page (int, optional, default 1), limit (int, optional, default 20), status (optional — Pending/Accepted/Rejected/Withdrawn)

status, when provided, filters to that single application status. Omitted, all of the adopter's applications are returned regardless of status.

Response

```
200 OK
{
  "success": true,
  "message": "Applications retrieved successfully",
  "data": [
    {
      "applicationID": 44,
      "petID": 9,
      "shelterID": 3,
      "applicationStatus": "Pending",
      "createdAt": "2026-08-03T09:00:00Z",
      "pet": { "petName": "Buddy", "petPhoto": "..." }
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 3, "totalPages": 1 }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Invalid status value	VALIDATION_ERROR	422

4. Visits

Endpoints for scheduling and viewing shelter visits or virtual meet-and-greets (Requirement A-07). Distinct from ADOPTION_APPLICATION — a visit can exist independently of a formal application, and a pet or assisting staff member on a visit may be null.

4.1 POST /api/v1/adopters/me/visits

Schedules a shelter visit or virtual meet-and-greet for the current adopter.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Body fields	shelterID (int, required), petID (int, optional), visitTime (ISO 8601 datetime, required), remarks (string, optional, ≤300 chars)

petID is optional — a visit is not required to be tied to a specific pet (e.g. a general shelter tour). visitTime must be in the future; staffID is left null at creation and assigned later by shelter staff.

Response

```
201 Created
{
  "success": true,
  "message": "Visit scheduled successfully",
  "data": {
    "visitID": 21,
    "adopterID": 12,
    "petID": 9,
    "staffID": null,
    "shelterID": 3,
    "visitTime": "2026-08-10T14:00:00Z",
    "remarks": "...",
    "visitStatus": "Confirmed"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing shelterID or visitTime	VALIDATION_ERROR	422
visitTime is in the past	VALIDATION_ERROR	422
Shelter or pet does not exist	NOT_FOUND	404

4.2 GET /api/v1/adopters/me/visits

Returns the current adopter's scheduled visits.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Query params	upcoming (boolean, optional) — when true, returns only visits with visitTime in the future and visitStatus of Confirmed

Response

```
200 OK
{
  "success": true,
  "message": "Visits retrieved successfully",
  "data": [
    {
      "visitID": 21,
      "petID": 9,
      "shelterID": 3,
      "visitTime": "2026-08-10T14:00:00Z",
      "visitStatus": "Confirmed",
      "pet": { "petName": "Buddy" },
      "shelter": { "shelterName": "..." }
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

Not paginated: an individual adopter's visit list is expected to stay small (dozens, not hundreds), unlike the shelter-wide staff view of all visits which will need pagination when built in Sprint 4.

5. Favorites

Endpoints for saving and removing pets from the adopter's favorites list (Requirement A-04). Backed by the FAVORITES junction table (adopterID, petID).

5.1 POST /api/v1/pets/:id/favorites

Adds a pet to the current adopter's favorites list.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Path params	id (int, required) — petID

Response

```
201 Created
{
  "success": true,
  "message": "Pet added to favorites",
  "data": { "adopterID": 12, "petID": 9 }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Pet does not exist	NOT_FOUND	404
Pet already in favorites	CONFLICT	409

Uniqueness: FAVORITES has a composite primary key on (adopterID, petID) — the same check-then-insert-with-DB-backstop pattern applies here for free, since a composite PK is itself a uniqueness constraint. A duplicate insert throws P2002, caught and returned as 409.

5.2 DELETE /api/v1/pets/:id/favorites

Removes a pet from the current adopter's favorites list.

Request

Field	Value
Method	DELETE
Auth required	Yes — Adopter only
Path params	id (int, required) — petID

Response

```
200 OK
{
  "success": true,
  "message": "Pet removed from favorites",
  "data": null
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Pet not in favorites (nothing to remove)	NOT_FOUND	404

5.3 GET /api/v1/adopters/me/favorites

Returns the current adopter's full list of favorited pets.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Favorites retrieved successfully",
  "data": [
    {
      "petID": 9,
      "petName": "Buddy",
      "petPhoto": "...",
      "adoptionStatus": "available",
      "shelterID": 3
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

Response shape mirrors the public pet catalog card fields (see 08-Public_API_Reference), not the full PET record — this list is meant to render the same PetCatalogCard component used elsewhere in the frontend.

6. Adopted Pets

Endpoints for an adopter to view pets they have adopted or fostered, including each pet's universal health passport (medical history and vaccination records that persist across inter-shelter transfers).

6.1 GET /api/v1/adopters/me/adopted-pets

Returns the pets currently or previously adopted/fostered by the current adopter, identified via ADOPTION_APPLICATION records with applicationStatus = 'Accepted' joined to PET.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Adopted pets retrieved successfully",
  "data": [
    {
      "petID": 9,
      "petName": "Buddy",
      "breedID": 4,
      "petDOB": "2023-02-14",
      "petWeight": 12.4,
      "petHeight": 45.0,
      "petBGroup": "A",
      "petColor": "Brown",
      "petSize": "Medium",
      "petPhoto": "...",
      "petSex": "M",
      "petDesc": "...",
      "microchipID": "982000123456789",
      "adoptionStatus": "adopted"
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

Response shape matches GET /pets/:id (see 08-Public API Reference) rather than a slimmed-down summary — the adopted-pets view needs the same full profile fields the public catalog detail view already exposes (age computed live from petDOB, weight, blood group, microchip, etc.), so the same shape is reused rather than defining a parallel one.

6.2 GET /api/v1/adopters/me/adopted-pets/:petId/vaccinations

Returns the vaccination history for one of the current adopter's adopted/fostered pets — part of the pet's universal health passport.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path params	petId (int, required)

Ownership check: the handler verifies an Accepted ADOPTION_APPLICATION exists linking this adopter to this pet before returning any vaccination data — an adopter cannot view vaccination records for a pet they have not adopted, even if they know the petId.

Response

```
200 OK
{
  "success": true,
  "message": "Vaccination records retrieved successfully",
  "data": [
    {
      "recordID": 15,
      "petID": 9,
      "vaccineID": 2,
      "vaccine": { "vaccineName": "Rabies", "manufacturer": "..."},
      "administeredDate": "2026-01-10T00:00:00Z",
      "dueDate": "2027-01-10T00:00:00Z",
      "administeredBy": 5,
      "administeredAt": 3
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Pet does not exist	NOT_FOUND	404
Adopter has not adopted this pet	FORBIDDEN	403

Response shape matches the VACCINATION_RECORD table directly, joined to VACCINE for display fields (vaccineName, manufacturer) — this is the same shape a future vet-facing endpoint would use to write these records, kept consistent rather than reshaping per-consumer.

This is read-only for adopters. Recording or updating a vaccination is restricted to the Vet role (see 06-Auth System / API Design — POST and PATCH on /appointments/:id/vaccinations), consistent with NF-02 RBAC.